

Data Storage Systems

Sistemas de Armazenamento de Dados

I/O Devices



I/O Devices

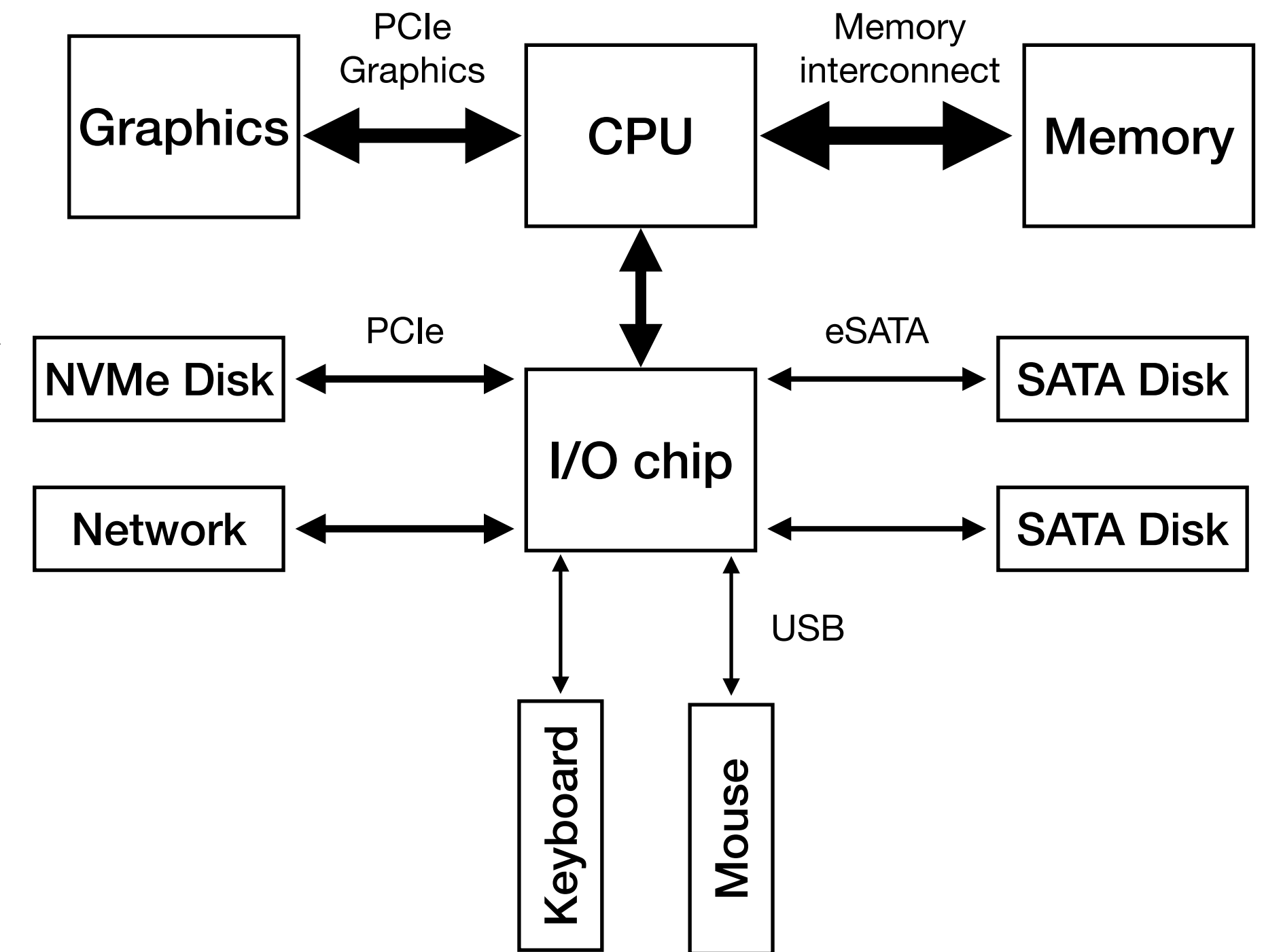
Brief overview of an intel-based architecture

- ◎ **CPU** and **memory** are connected by the **fastest BUS**

- A **high-speed BUS** is also used for **graphic cards**

- ◎ **CPU** connects to an **I/O chip** through a **Direct Media Interface (DMI)**

- **PCIe** is the fastest I/O BUS, used for network and modern storage disks (e.g., NVMe SSDs)
- **eSATA** is used by more traditional disks (SATA SSDs, HDDs)
- The **USB** BUS is used for low-performance devices such as the keyboard, the mouse, external drives ...

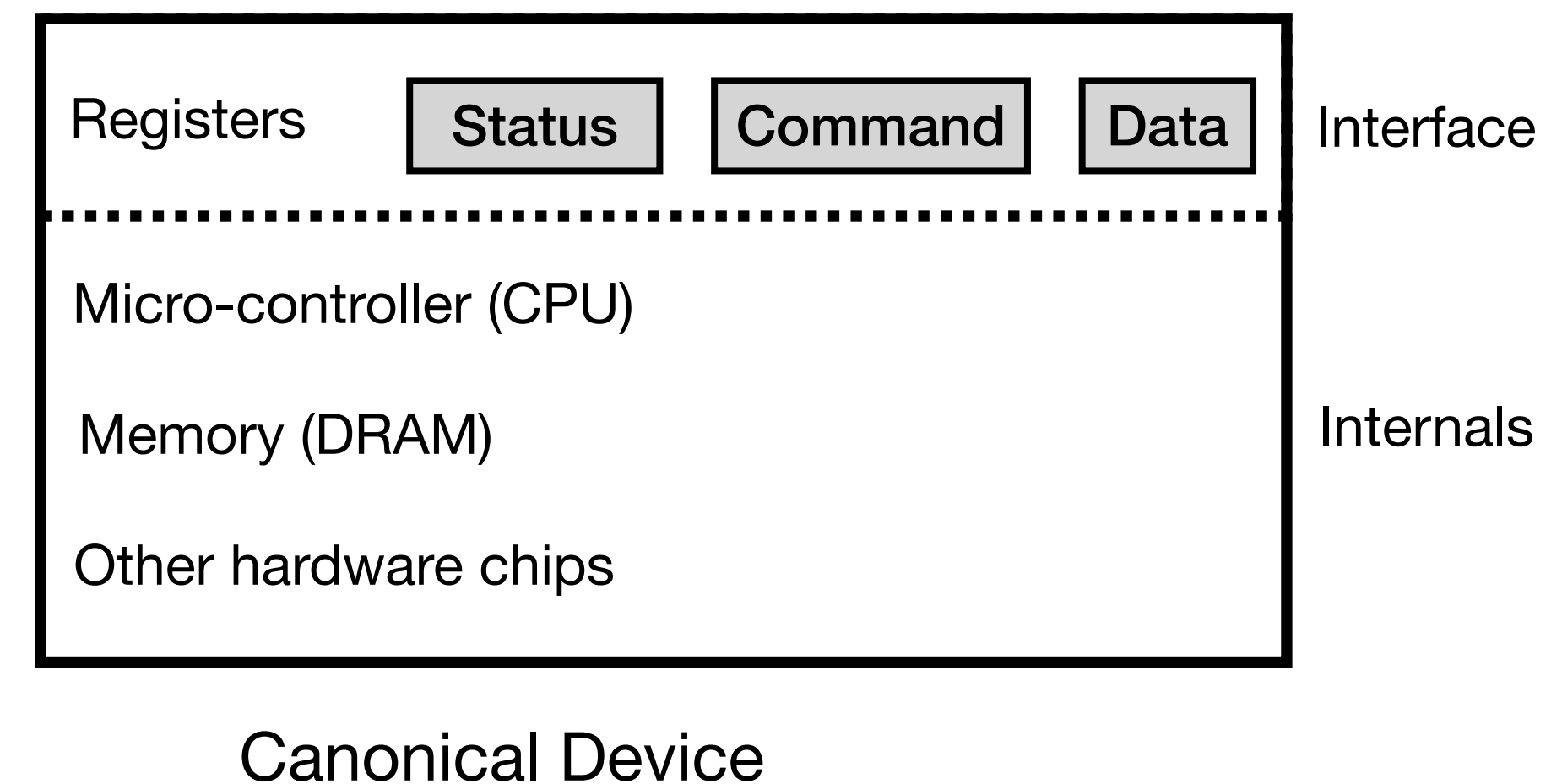


Modern system architecture
(Intel-based)

I/O Devices

A canonical device

- A device has two main components
 - The **interface** presented to the system
 - Its **internal structure**, such as the hardware chips (e.g., simple CPU, memory, ...) and firmware (i.e., the software that runs in the device)
- In a canonical device (not a real device)
 - The device is controlled through three registers that are read and/or written by the OS
 - **Status:** holds the current status of the device
 - **Command:** holds commands to be executed by the device
 - **Data:** holds data to be written to or read from the device



I/O Devices

A canonical device

- ◎ *A simple protocol* to interact with (write to) a disk device would be
 1. The OS waits for the device to be ready by **repeatedly reading the status register**. We call this **polling** the device (i.e., repeatedly asking the device)
 2. The OS writes some bytes in the **data register**. When the CPU is responsible for copying data to this register, we refer to this approach as **programmed I/O (PIO)**
 3. The OS writes the command register to let the device know that data is present and should persist it
 4. Finally, the OS waits for the request to finish by **polling** again the device (status register)
- ◎ For the OS to interact with the device, two approaches are possible
 - **Specific I/O instructions** to write to the **registers** are supported by the OS
 - **Memory-mapped I/O** is used, i.e., the device registers are mapped as these were loaded into memory regions, and the OS uses these as regular memory

I/O Devices

Direct Memory Access (DMA)

- Using **programmed I/O (PIO)** forces the CPU to stop other tasks just to copy data from/to memory to/from the I/O device
- The **DMA** engine is a specific device that can orchestrate data between the I/O devices and main memory with little CPU intervention
- The OS just tells the DMA device(s) where data is to be copied from and to, and then the CPU is free to do other tasks

I/O Devices

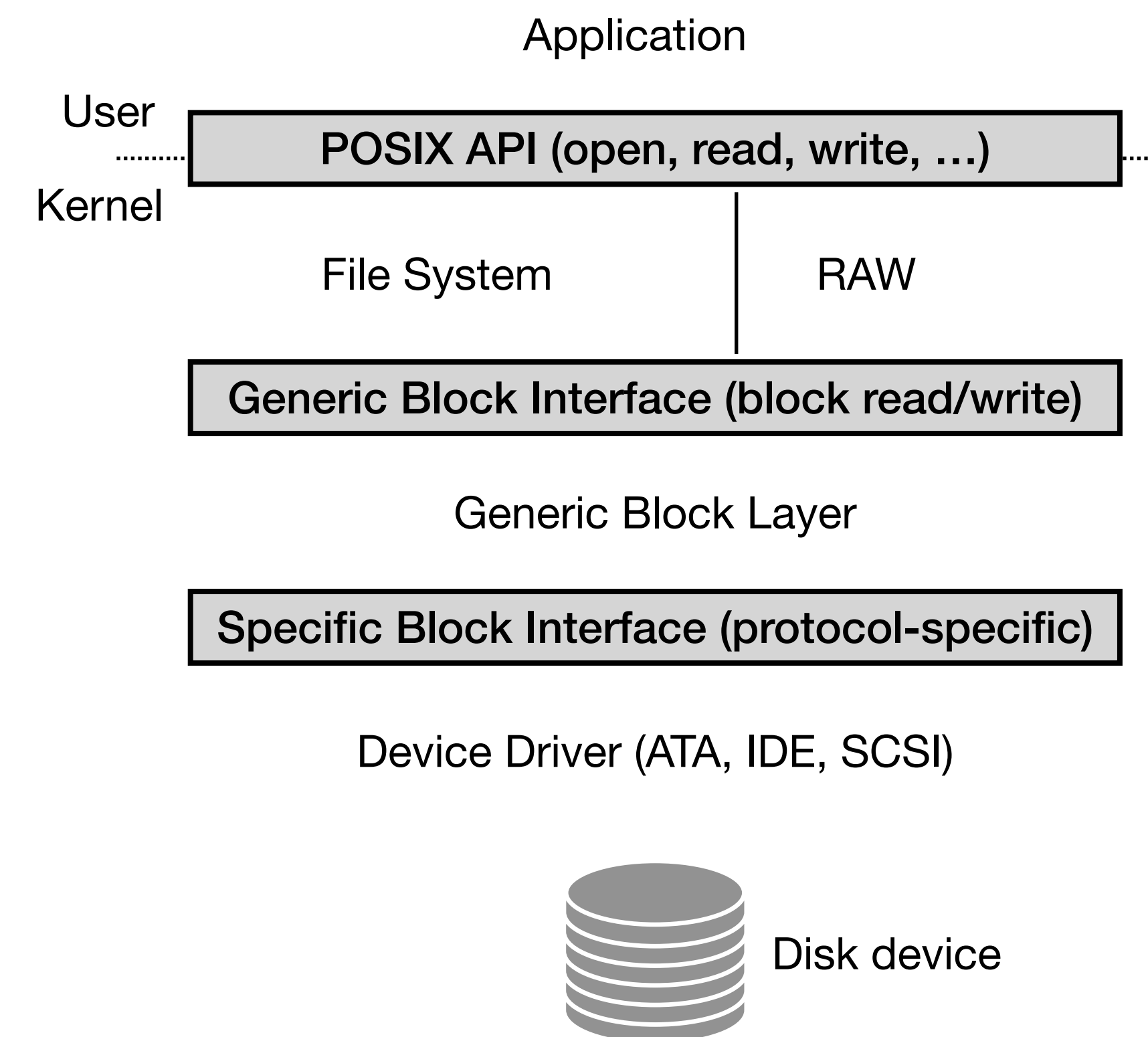
Polling vs interrupts

- Polling seems inefficient; it wastes CPU cycles waiting for the device to complete its activity
 - Remember **interrupts**? The device raises a **hardware interrupt** when the request is ready meanwhile, the OS (and the CPU) can do other useful tasks
- However, using interrupts may not always be the best decision (strange, right?)
 - If the **device is very fast** (i.e., it answers back under one (or a few) poll requests), the performance cost of **context switching** and **handling interrupts** may be spared
- When the **speed of the device** is **unknown** or **varies over time**, a hybrid approach is used
 - Do some poll requests, but if the device is slow in serving the request, go for interrupts
- Devices supporting parallel requests often **coalesce** (batch) multiple interrupts before notifying the OS (i.e., to avoid the cost of handling these independently)
 - Trade-off between **throughput** and **latency** (a typical trade-off when requests are batched)

Fitting Into The OS Storage Stack

A traditional (and simplified) Linux I/O stack

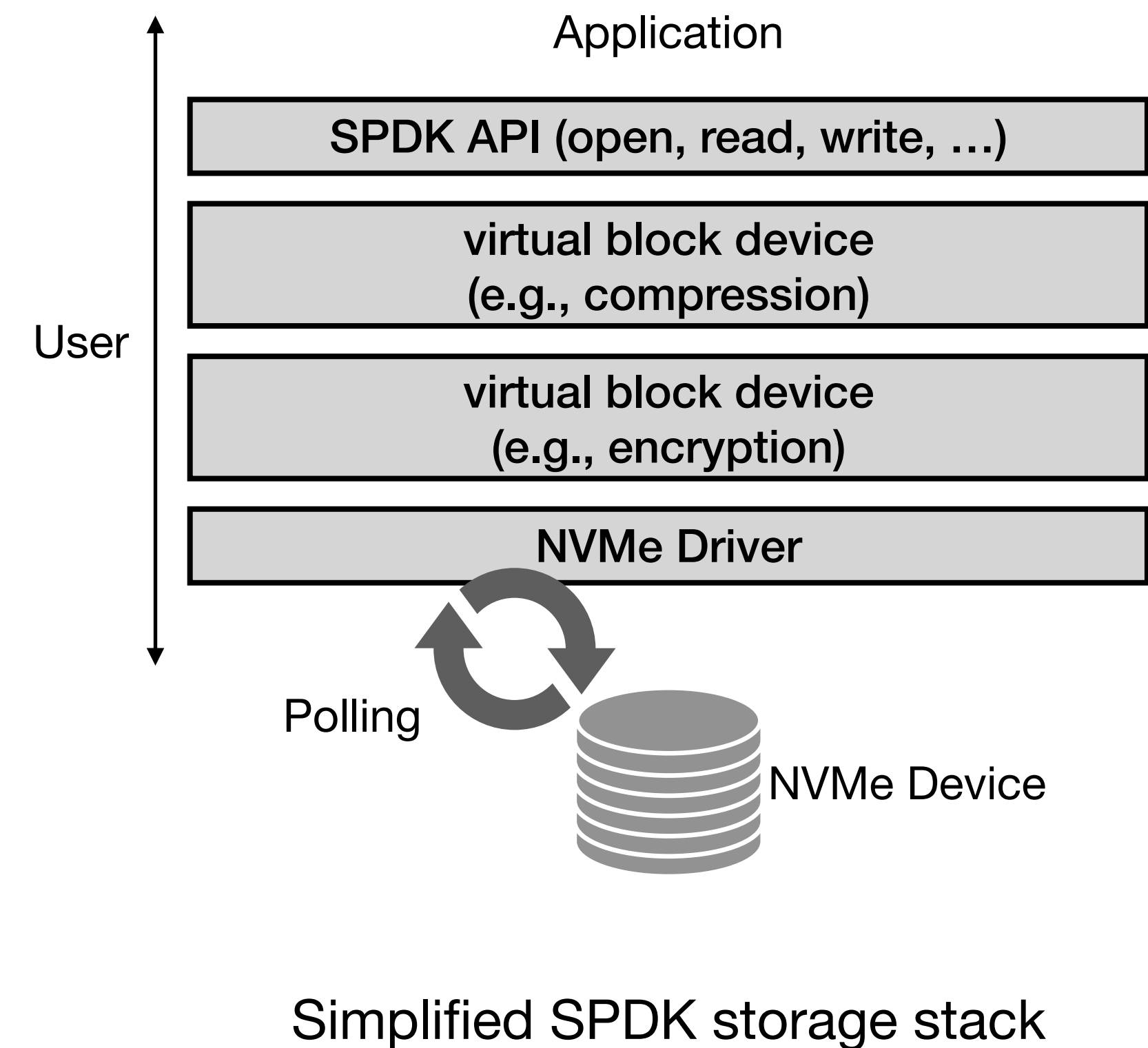
- **Applications** access a file system or a RAW block device, exposing the POSIX API
- The **file system** (and applications using a raw block device) only sees the generic interface exposed by the block layer (i.e., block read and write requests)
- The **block layer** routes requests to the right **device driver**, which knows how to translate these to protocol-specific writes and reads at the I/O device
 - I/O devices support different interfaces (e.g., ATA, IDE, SCSI, PCIe), which require distinct device drivers



A Non-Traditional Storage Stack

Storage Performance Development Kit (SPDK)

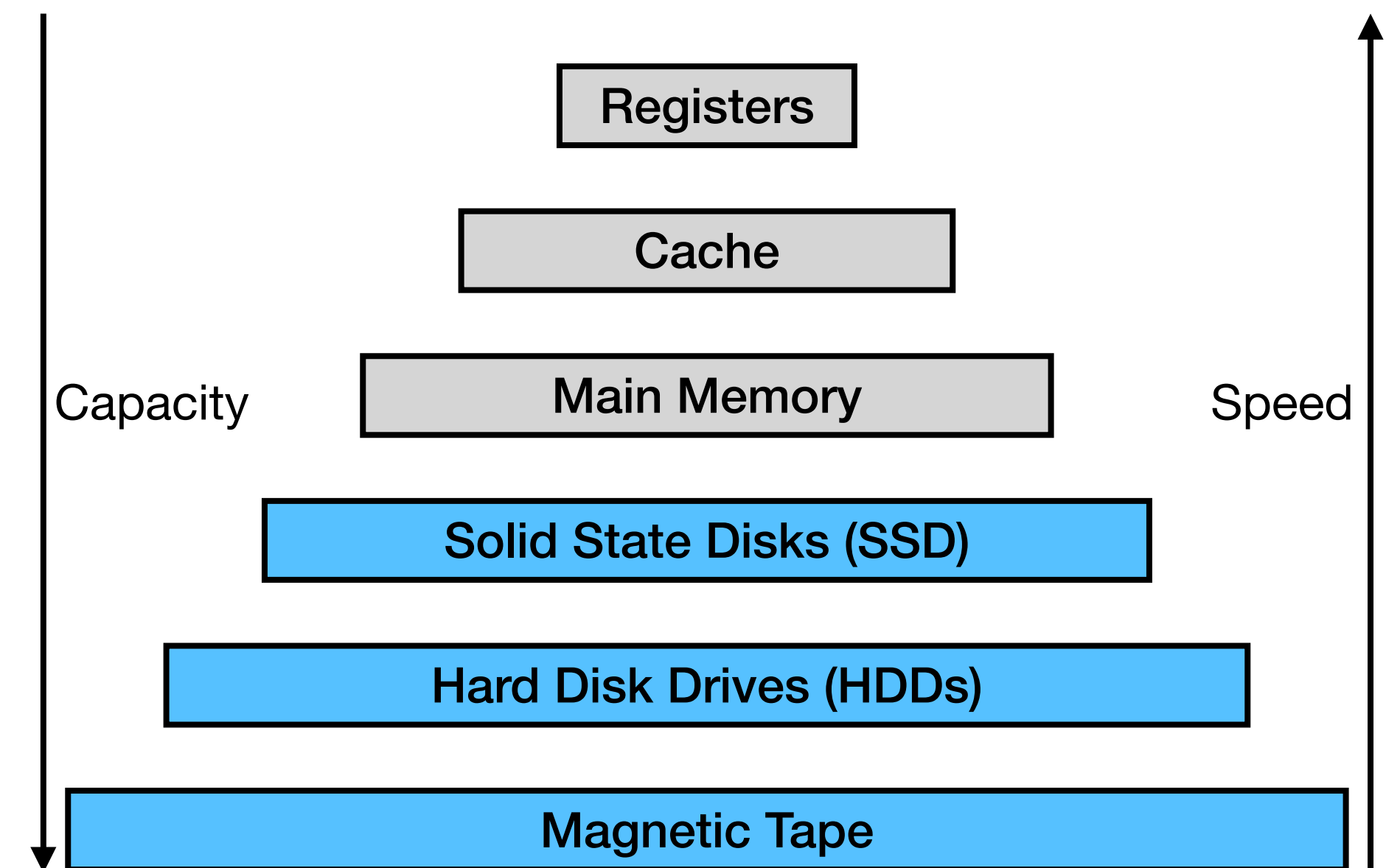
- Set of tools and libraries for writing high-performance, scalable, user-mode storage applications
- **NVMe driver** implemented completely in **user space**
 - The OS (kernel) is bypassed, avoiding system calls and enabling zero-copy access from the application (DMA support)
- **Polling** is used instead of interrupts - lowers latency!
- Relies on **message passing** across threads instead of locking
- Offers a **stackable design** of virtual block devices (bdevs) offering different features (e.g., compression, encryption)
- Supports **block devices over fabrics** (Ethernet, Infiniband, Fibre)
 - Uses RDMA for OS bypassing when accessing other computers' memory



Persistent Storage Devices

Which disk should I pick?

- ◎ CPU registers, caches, and main memory have the fastest access times (in this order), but are **volatile**
- ◎ **Persistent** devices such as SSDs, HDDs, and Tapes have slower access speeds (in this order)
- ◎ **Trade-off:** As speed increases, the cost per byte is higher
- ◎ Given the sheer amount of persistent storage devices, which one(s) should we pick?
 - We need to understand their strengths and drawbacks i.e., performance, cost, capacity, ...
 - And to know the **workload** (i.e., the type of requests) that these devices will be serving¹



¹Check the slides at the end for further info about some typical storage workloads

Persistent Storage Devices

And the main workloads these target...

● Tape

- Used for archival data
- Reliable and cheap
- Limited support for random accesses and in-place updates

● HDD

- Used for archival, backup, and (still in some cases) primary data
- Still cheap, with support for random accesses and in-place updates

● SSD (includes SATA/NVMe SSDs)

- Used for backup and primary data
- More expensive than HDDs but faster, especially for random read and write accesses

● Persistent Memory¹

- Used for primary storage (mostly used as cache²)
- Speed closer to RAM for sequential and random workloads, but expensive

¹ Intel Persistent Memory is now discontinued

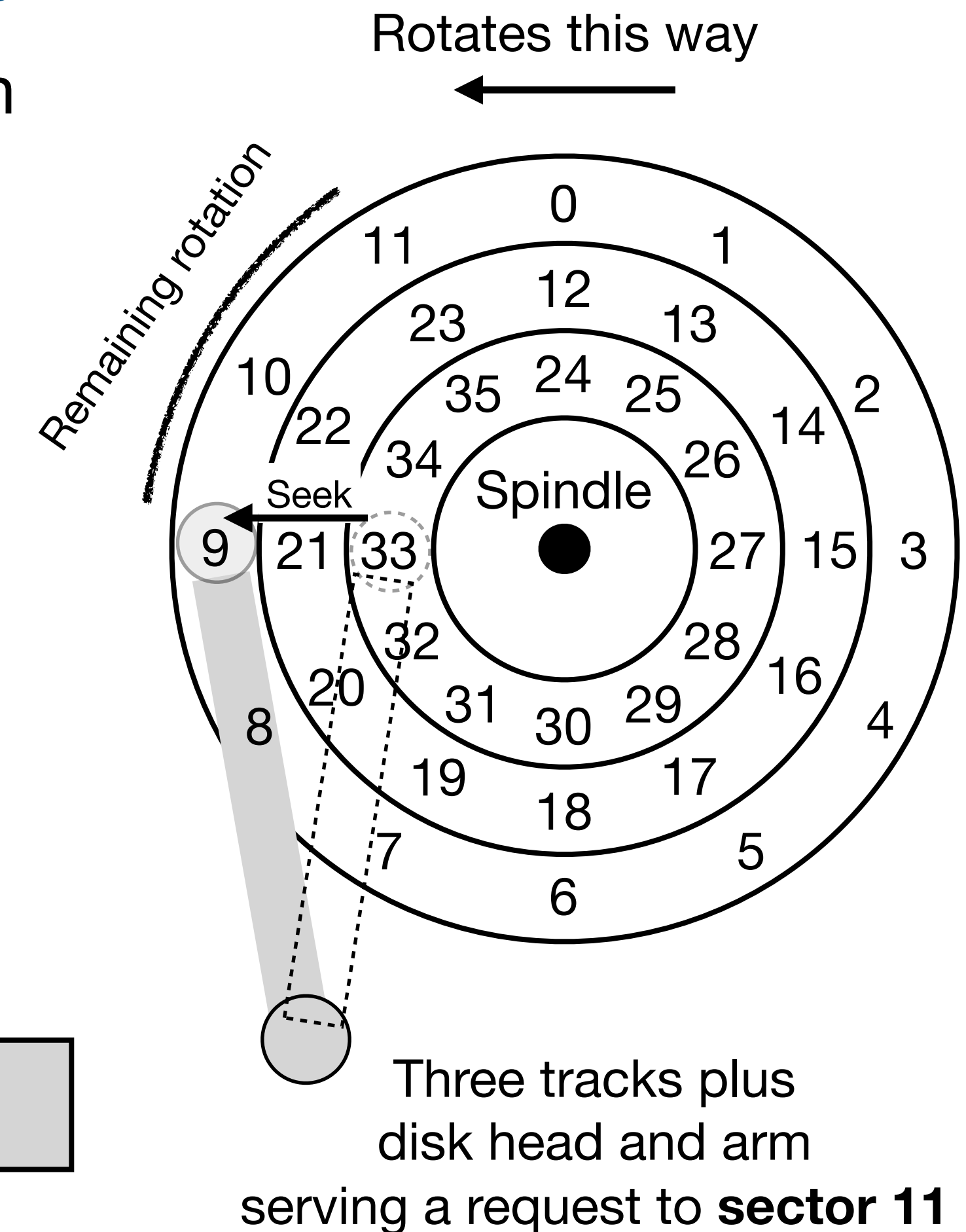
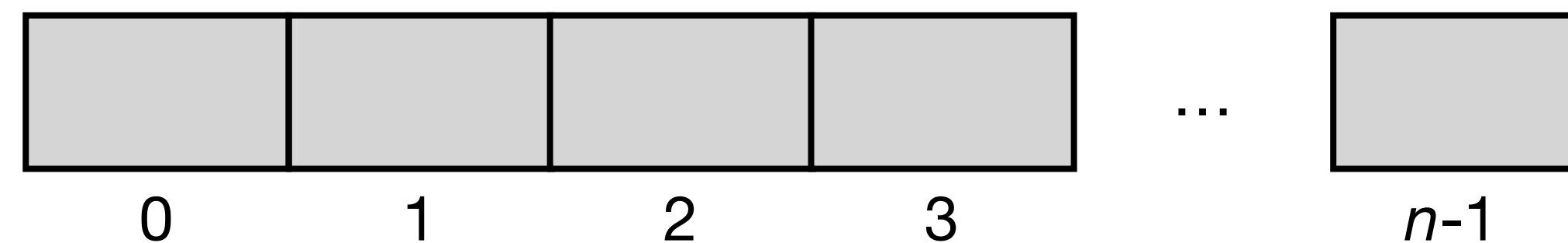
² Don't forget that RAM (main memory) is also used as cache for persistent data :)

Hard Disk Drives (HDDs)

Logical view and geometry

- Composed of **sectors** (typically 512 bytes) that can be read or written
 - A disk can be logically seen as an array of sectors (0 to $n-1$)
- Geometry-wise, data is encoded on each surface of a **platter** in concentric circles (**tracks**), each containing multiple **sectors**¹
 - The rate for modern drives is in the 7200 to 15000 **rotations per minute (RPM)**
- The I/O time to read/write a given sector depends on
 - the time to **move the disk arm** to the right track
 - the time to **place the disk head** under the right sector
 - the time to **read/write** (transfer) the sector

Logical representation of a disk
(an array of sectors 0 to $n-1$)



¹Check the slides at the end for further info about HDDs geometry and I/O handling

Hard Disk Drives (HDDs)

Some considerations

◎ Workloads

- **Sequential:** Reading a large number of contiguous sectors per I/O request only requires a single seek and rotation before a long transfer time
- **Random:** Issuing I/O operations to non-contiguous sectors requires a seek, rotation, and transfer per request
- **Conclusion:** Whenever possible, transfer data to and from HDDs sequentially. When such is not possible, do random accesses with large payloads (block sizes)

◎ **Multi-sector operations** are possible

(e.g., many file systems read or write 4KB (or more) at a time)

- However, the drive manufacturers only guarantee that a single 512-byte write is **atomic** (i.e., it will either complete in its entirety or it won't complete at all)
- If a power loss occurs, only a portion of a larger write may be completed (**torn write**)

◎ Disks usually have a **small cache** (e.g., 8, 16, 32 MB) to hold data read from or written to disk

- **Important:** When should the disk acknowledge the completion of a write request?
 - When it is in the cache (**write-back**) or when it has actually been written to the disk platter (**write-through**)?

Scheduling Policies for HDDs

Disk scheduler

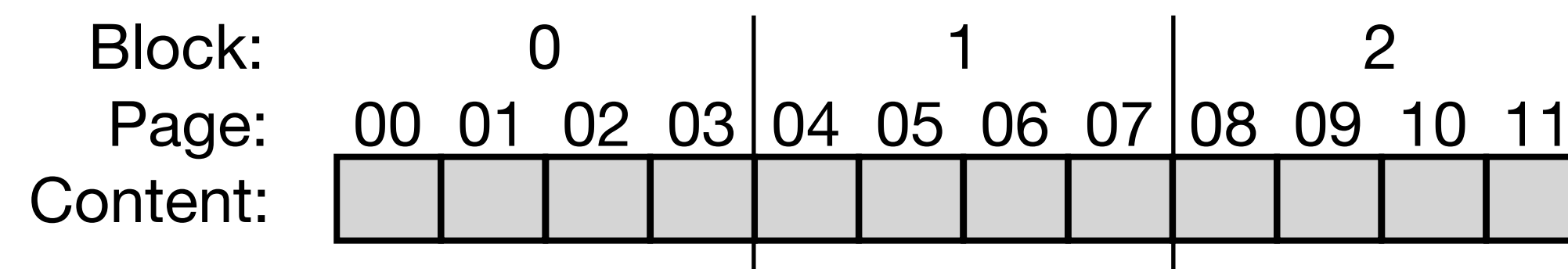
- In older systems, disk scheduling was handled by the OS's disk scheduler. Remember the **HDD scheduling policies** (OS classes)¹?
 - **SSTF**: Serve first the requests for the nearest track (**shortest seek time**)
 - **SCAN/C-SCAN**: **Sweep the disk** from inner to outer tracks (or the other way around)
 - **SPTF**: Consider **rotational** and **seek** time to choose the shortest positioning time
- In modern systems, the OS picks a few requests and lets the disk controller decide the best way to schedule them (e.g., by following SPTF)
 - The OS still implements important optimizations such as **I/O merging** *i.e.*, requests to contiguous sectors are merged before being sent to disk

¹Check the slides at the end for further info about each scheduling policy

Solid State Drives (SSDs)

Brief overview

- ◎ After decades of dominance of HDDs, a new form of persistent storage gained traction, the **Solid-State Drives (SSDs)**
 - The technology underlying these is known as **flash**, more specifically, NAND-based flash
- ◎ SSDs have no mechanical moving parts, unlike HDDs. Bits are stored on transistors
 - No arm seek time or head rotation time!
 - No mechanical errors (e.g., head crashes)
- ◎ The disk is organized into flash chips, each containing two different-sized units
 - **Blocks** (also called **erasure blocks**), which are typically of size 128KB or 256KB
 - Inside each block, there are multiple **pages**¹, each with a few KBs in size (e.g., 4KB)



A simple flash chip with 3 blocks, each with 4 pages

¹ Do not confuse the term page here with memory pages. CS people tend to use the same term, i.e., page, block, ..., for different contexts

Solid State Drives (SSDs)

Interesting characteristics and challenges

- ◎ Flash chips allow **reading single pages very efficiently**, regardless of the position of the page on the device and the location of the previous request
 - **Good for random read workloads** (unlike HDDs)!
- ◎ Before writing/updating a page, the **full block** containing that page must be **erased**
 - **Expensive** operation **performance-wise**, that erases the content of all pages at the block (i.e., the content of valid pages must be copied somewhere else before erasing their block)
- ◎ Once a block has been erased, the corresponding pages can be **programmed** (i.e., written with new content)
 - More efficient (time-wise) than erasing a block, but less efficient than reading a page

Solid State Drives (SSDs)

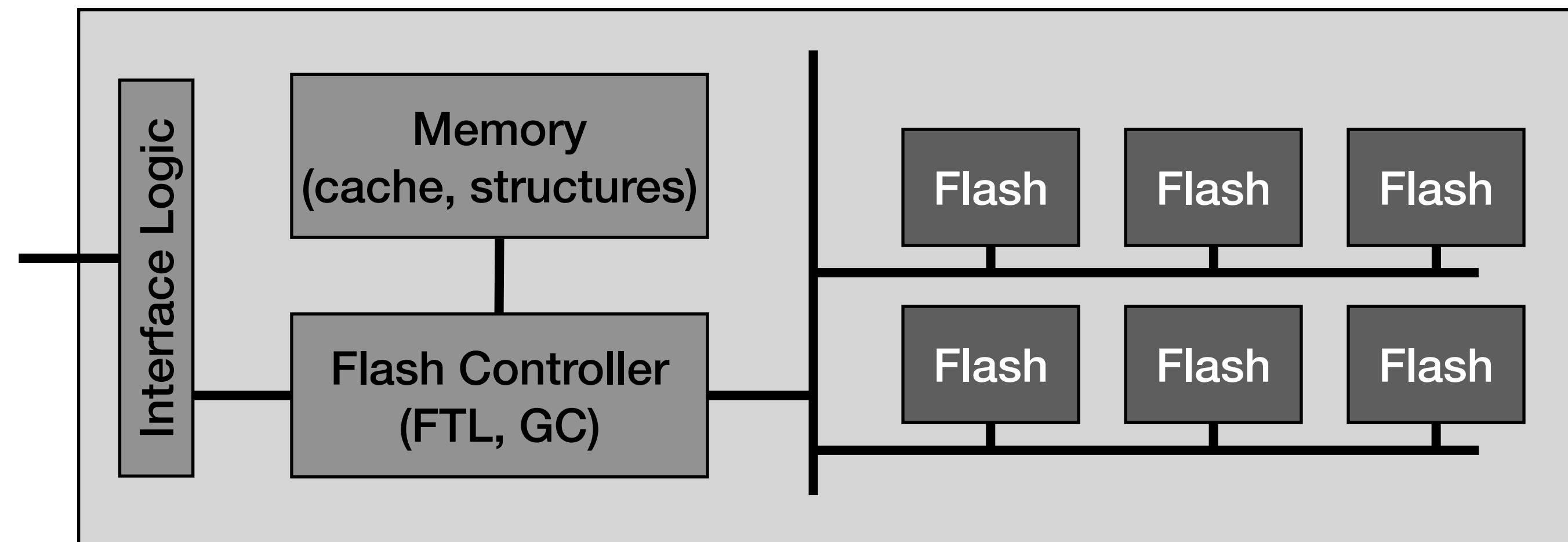
Wear out and disturbance

- Erasing a block also **wears out** the lifespan of the SSD drive
 - Each block can only be erased and programmed a certain number of times (e.g., 10000 times)
 - This challenge motivated several **wear leveling algorithms** that spread erase/program operations (i.e., write load) across all the blocks of the device evenly to extend its lifespan
- Also, sometimes, when programming (writing to) a page within a flash chip, neighboring (higher) pages may have some bits flipped, a phenomenon known as **disturbance**
 - Pages in a block must be programmed in order

Solid State Drives (SSDs)

Providing the disk abstraction

- How are multiple flash chips provided as a traditional storage device?
 - The SSD device offers a standard block (sector) interface atop the flash chips

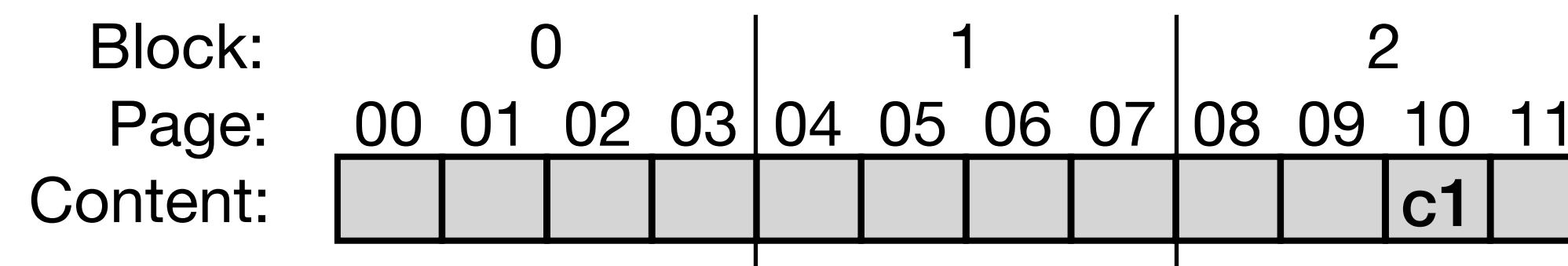


Logical Diagram of a Flash-based SSD

Flash Translation Layer (FTL)

Direct Mapping strategy

- The **FTL** maps **logical pages** to **physical ones** (flash chips)
- **Direct mapping:** Use a direct mapping between logical and physical pages
 - Writing logical page N may require **reading a full block of pages**, **erasing it**, and **programming** the block's old pages and the new one, leading to **write amplification**¹ and **bad performance**
 - Poor wear leveling approach (imagine the client's workload constantly rewrites the same data), leading to **several erasure operations** and **poor reliability**



Logical page 10, with content c1, is written to physical page 10

¹ **write amplification:** for each logical page written by the client, several flash pages may need to be written

Flash Translation Layer (FTL)

Log-Structured strategy

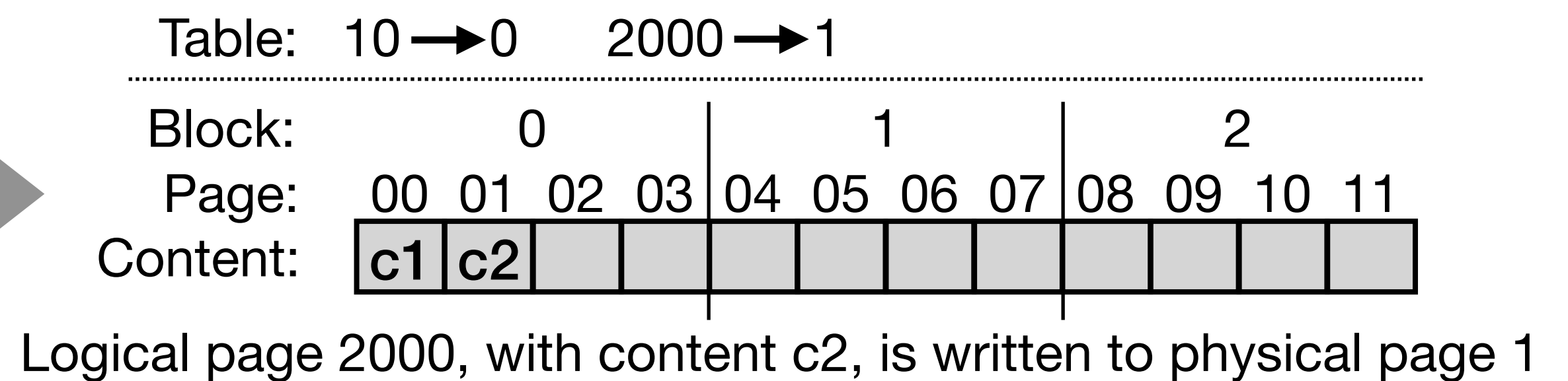
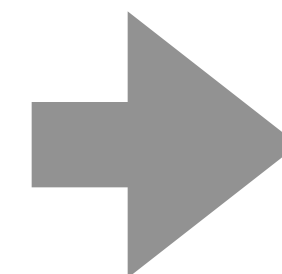
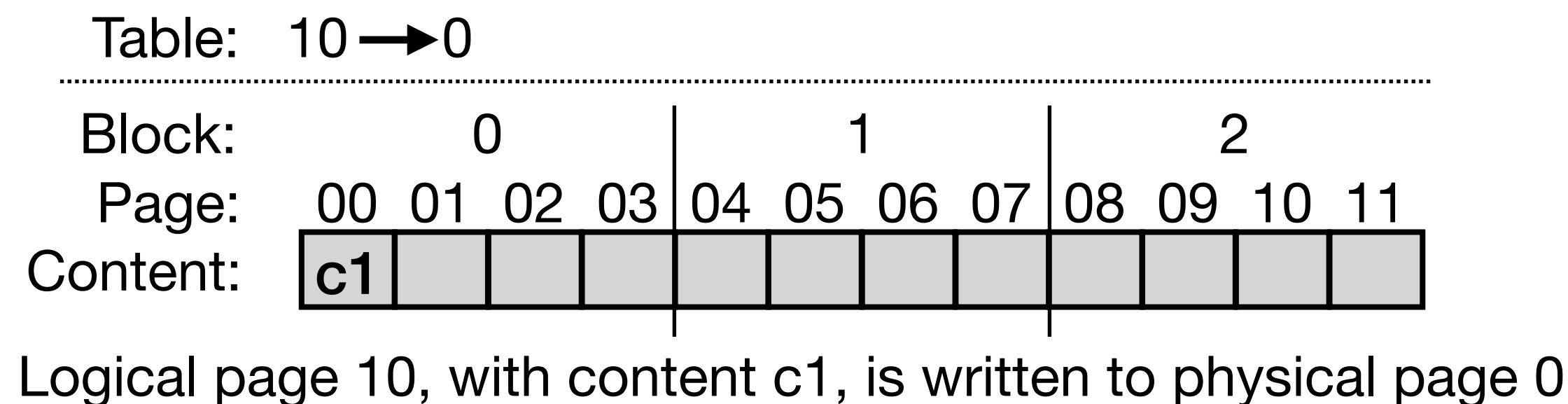
◎ **Log-Structured FTL:** A write to logical page N is appended to the next free spot in the currently-being-written-to flash block

- Better **wear leveling** and **random write** performance (unlike HDDs)!

◎ A mapping table (index) between logical and physical pages is needed

- Different mapping strategies: **per page** or **per block** of contiguous logical pages¹
- Do you have ideas on the advantages and disadvantages of each mapping strategy?

What about a **hybrid one**? Use both mappings and build something similar to a sparse index¹



¹ Check Section 44.9 for more info - Operating Systems: Three Easy Pieces. Arpaci-Dusseau Books, 2018

Garbage Collector

Log-Structured strategy

- With a Log-Structured FTL, page updates result in writing a new page and leaving an unused one
- The **Garbage Collector (GC)** periodically tracks **unused pages** and **erases blocks**
 - Sometimes, this requires copying pages still in use, at the block to be erased, to other blocks
 - Must follow a clever algorithm for good wear leveling (*i.e.*, ensure all SSD blocks wear at roughly the same time)
- **Data structures** for the FTL and GC, along with **data caches**, are kept on volatile memory (SRAM) at the device

Table: 10 → 0 2000 → 1

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	c1	c2										

Initial mapping

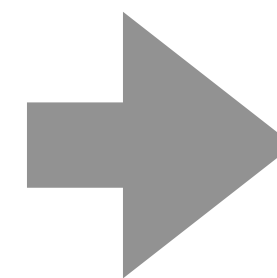


Table: 10 → 2 2000 → 1

Block:	0				1				2			
Page:	00	01	02	03	04	05	06	07	08	09	10	11
Content:	c1	c2	c3									

Logical page 10, with content c3, is written to physical page 2
Physical page 0 (old content - c1) can be garbage collected

Summary

HDDs vs SSDs

- When compared to HDDs, SSDs
 - Excel in **random write and random read** performance
 - Provide better performance for **sequential workloads** (although the difference is smaller)
- Remember that performance gains typically increase **monetary costs**
- Although in SSDs, the gap between sequential and random workloads' performance is smaller, it still pays off (especially for I/O-demanding applications) to **reduce random I/O** operations

More Information

- **Chapters 36, 37, and 44** - Remzi H. Arpaci-Dusseau, Andrea C. Arpaci-Dusseau. **Operating Systems: Three Easy Pieces**. Arpaci-Dusseau Books, 2018.
- **SPDK documentation** - <https://spdk.io/doc/>
- **Understanding TLC Flash** - Kristian Vatto. AnandTech, September, 2012.
<http://www.anandtech.com/show/5067/understanding-tlc-nand>
- **The Unwritten Contract of Solid State Drives** - JunHe, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys'17
- **Understanding modern storage APIs: a systematic study of libaio, SPDK, and io_uring** - Diego Didona, Jonas Pfefferle, Nikolas Ioannou, Bernard Metzler, and Animesh Trivedi. SYSTOR'22

Questions?

Extra Material

Storage Workloads

Archival data

- ◎ Data is stored for archival purposes
i.e., data that is rarely accessed but may be relevant in the future
 - **Throughput** (*i.e.*, number of operations done per unit of time) is favored over latency (*i.e.*, time required to serve each operation)
 - Large amounts of data must be written/read efficiently
 - **Sequential** and **append-only** workloads
 - Archival data (e.g., files) is written and read sequentially
 - Archival data is usually append-only (*i.e.*, **write-once** data with no in-place updates)

Storage Workloads

Backup data

- ◎ Data backups of *fresh* data
i.e., data still in use and may be accessed frequently in the near future
 - **Throughput** is still favored over latency
 - Large amounts of data must be written/read efficiently
 - **Sequential** workloads mainly (random accesses may also happen)
 - Sometimes one may want to retrieve/update specific parts of backup data (e.g, specific files or even small portions of these)
 - **Data can now be updated** (typically in a sporadic fashion)
 - In some cases, **only diffs** (modified data) **are stored** across backups

Storage Workloads

Primary Storage¹ (not only RAM!)

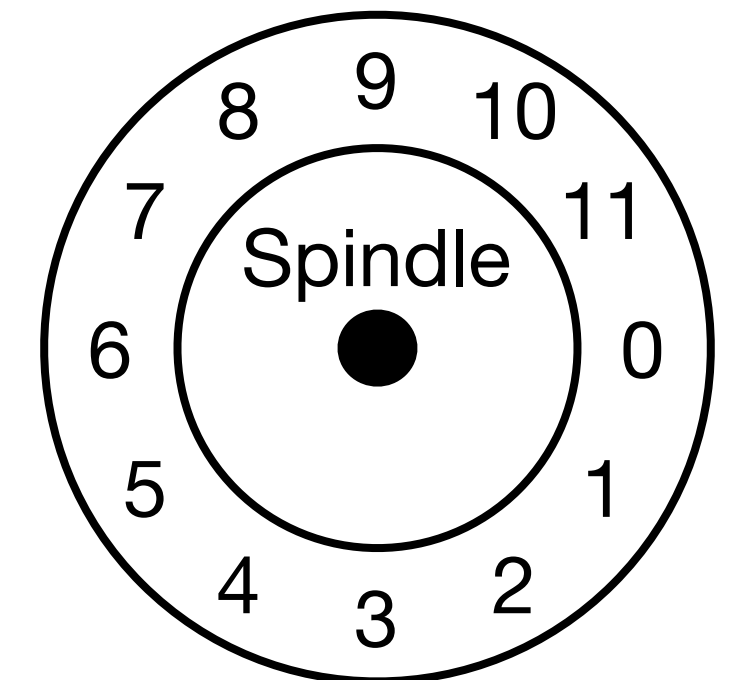
- ◎ Storage support for databases, data analytics, AI frameworks, VMs ...
 - **High-throughput** and/or **low-latency** are now desirable
 - Large amounts of data may be written/read (throughput)
 - Small-sized writes/reads must be done efficiently (latency)
 - **Sequential** and **random** workloads
 - Content (e.g., of files) may be partially accessed and out of order
 - **Data-** and **metadata-intensive** workloads
 - E.g., frequent access to the content (data) of files but also to their metadata
 - Data is expected to be **updated frequently**

¹Definition taken from: Paulo, J and Pereira, J. 2014. *A Survey and Classification of Storage Deduplication Systems*. ACM Computing Surveys

Hard Disk Drives (HDDs)

Geometry of an HDD

- ◎ The **platter** is a circular, hard surface on which data is stored
 - Each platter has two sides, each of which is called a **surface**
- ◎ Data is encoded on each surface in concentric circles of sectors named **tracks**
 - A single surface contains thousands and thousands of tracks!
- ◎ A drive may have one or more platters bound around a **spindle**. The spindle is connected to a motor that rotates the platters at a given fixed rate
 - The rate for modern drives is in the 7200 to 15000 **rotations per minute (RPM)**
 - Assuming 10000 RPM, a single rotation takes 6 milliseconds
- ◎ The **disk head** is used to read or write a specific sector (one head per surface)
 - The **disk arm** moves across the surface to position the head on the right track

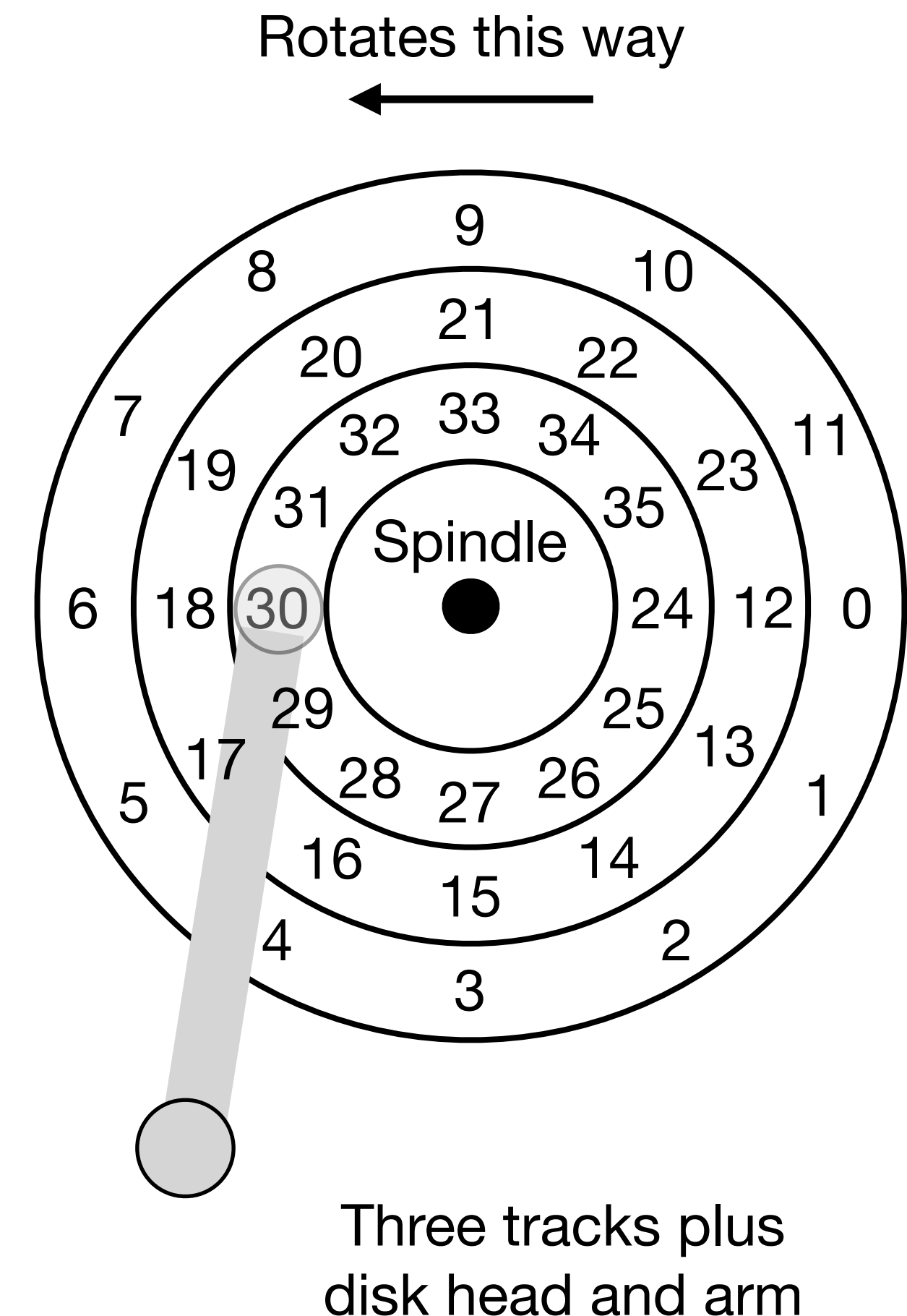


A disk with just a single track

Geometry of HDDs

A multiple track disk¹

- **Example:** disk with 3 tracks, each with 12 sectors
 - The disk head is placed at sector 30 (innermost track)
 - The platter rotates counterclockwise
- A request to **sector 11** is made
 1. The arm must first move to the outermost track (**seek** phase)

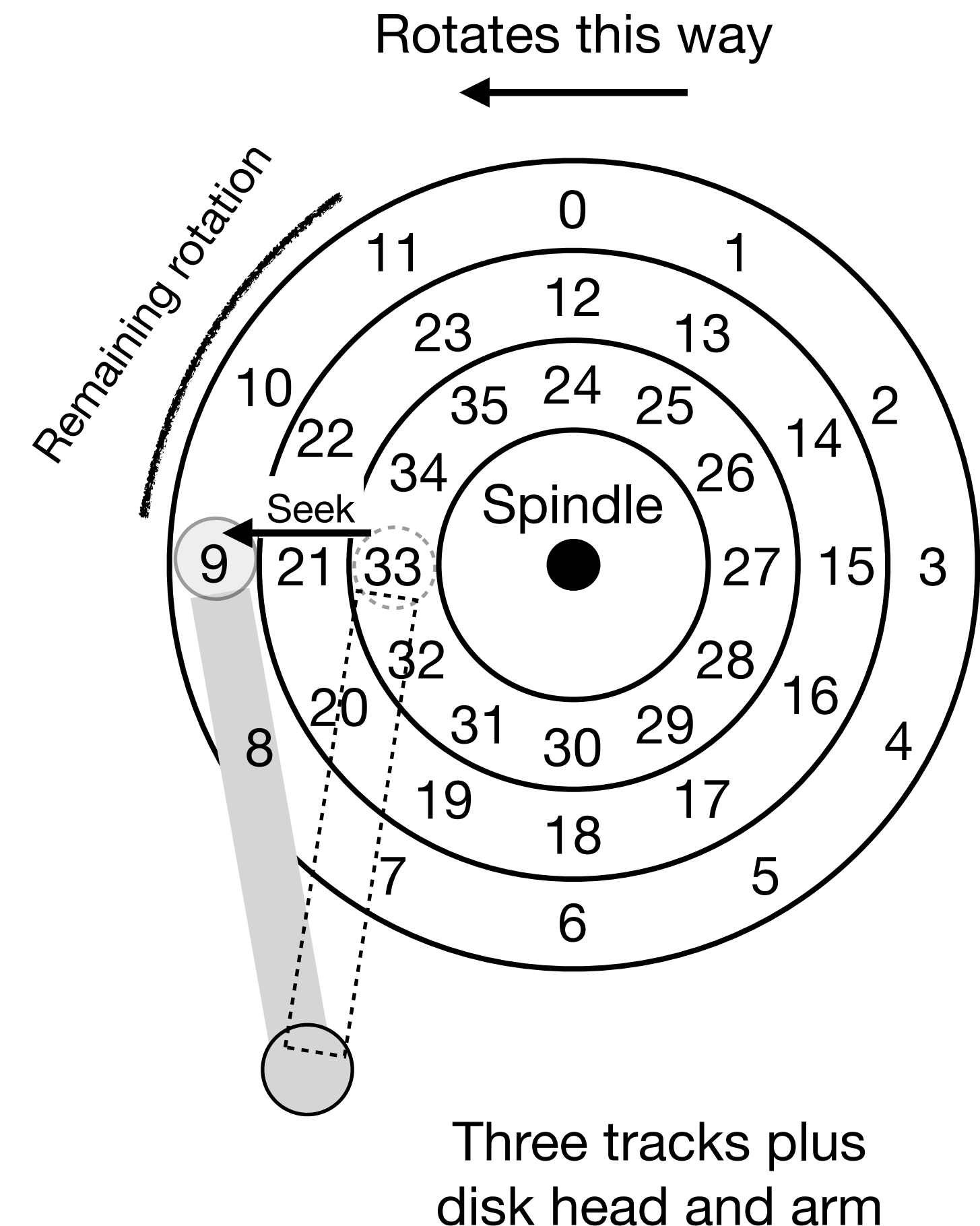


¹Check the slides at the end for further info about HDDs geometry

Geometry of HDDs

A multiple track disk¹

- **Example:** disk with 3 tracks, each with 12 sectors
 - The disk head is placed at sector 30 (innermost track)
 - The platter rotates counterclockwise
- A request to **sector 11** is made
 1. The arm must first move to the outermost track (**seek** phase)
 2. When the seek is done, the platter has rotated (e.g., the head is passing through sector 9). One must wait for sector 11 to be under the head (**rotation** phase)



¹Check the slides at the end for further info about HDDs geometry

Geometry of HDDs

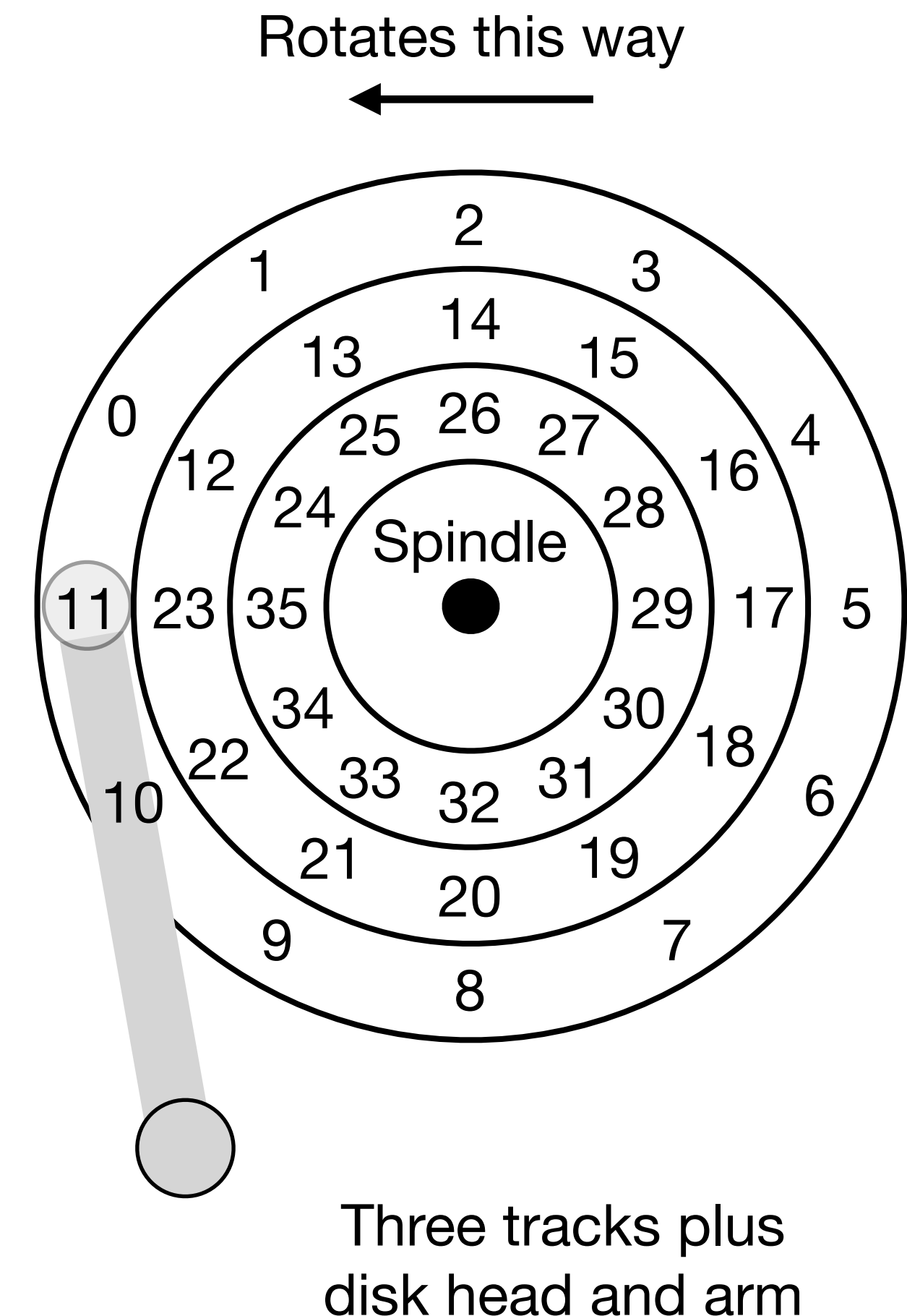
A multiple track disk¹

● **Example:** disk with 3 tracks, each with 12 sectors

- ▶ The disk head is placed at sector 30 (innermost track)
- ▶ The platter rotates counterclockwise

● A request to **sector 11** is made

1. The arm must first move to the outermost track (**seek** phase)
2. When the seek is done, the platter has rotated (e.g., the head is passing through sector 9). One must wait for sector 11 to be under the head (**rotation** phase)
3. When sector 11 is under the head, data can be read from or written to disk (**transfer** phase)

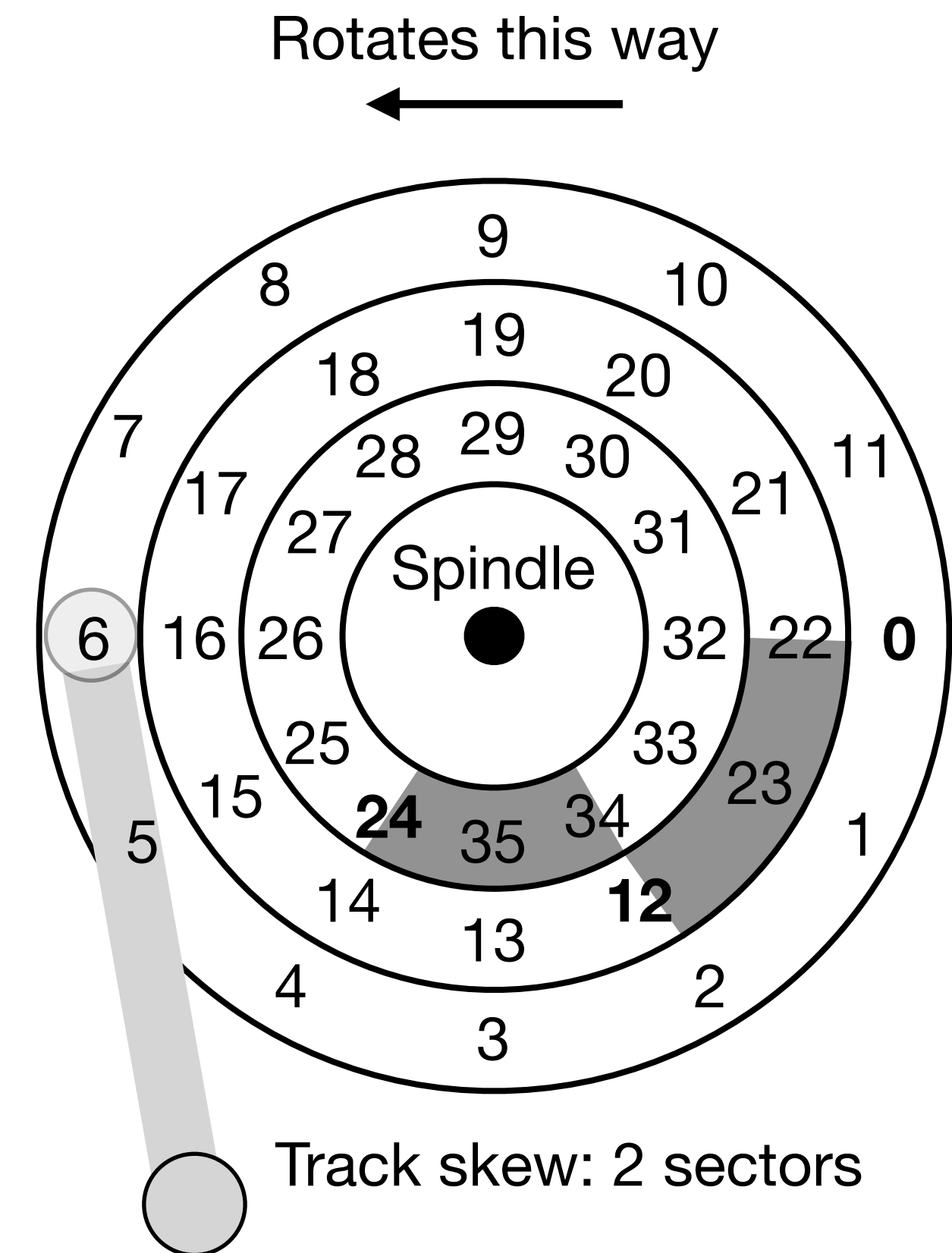


¹Check the slides at the end for further info about HDDs geometry

Geometry of HDDs

Some considerations

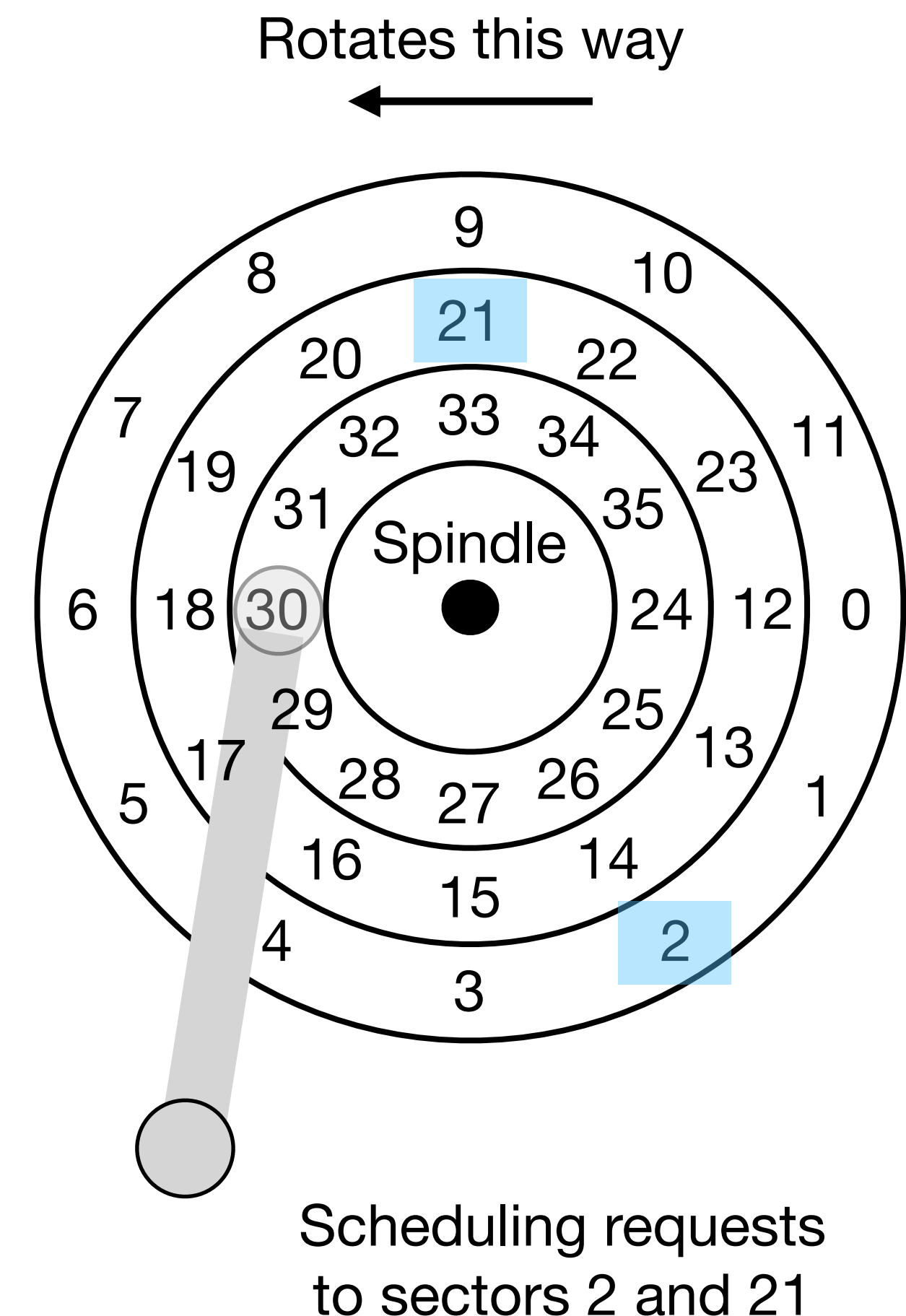
- To ensure that sequential operations are properly served when crossing track boundaries, sectors of tracks may be **skewed**
 - **Compensates seek time**, i.e. when the head reaches the right track, the desired sector has not rotated under the head yet
- Outer tracks tend to have more sectors than inner ones (more space available) and can be organized into zones (**multi-zoned** disk drives)



Scheduling Policies for HDDs

Shortest Seek time First (SSTF)

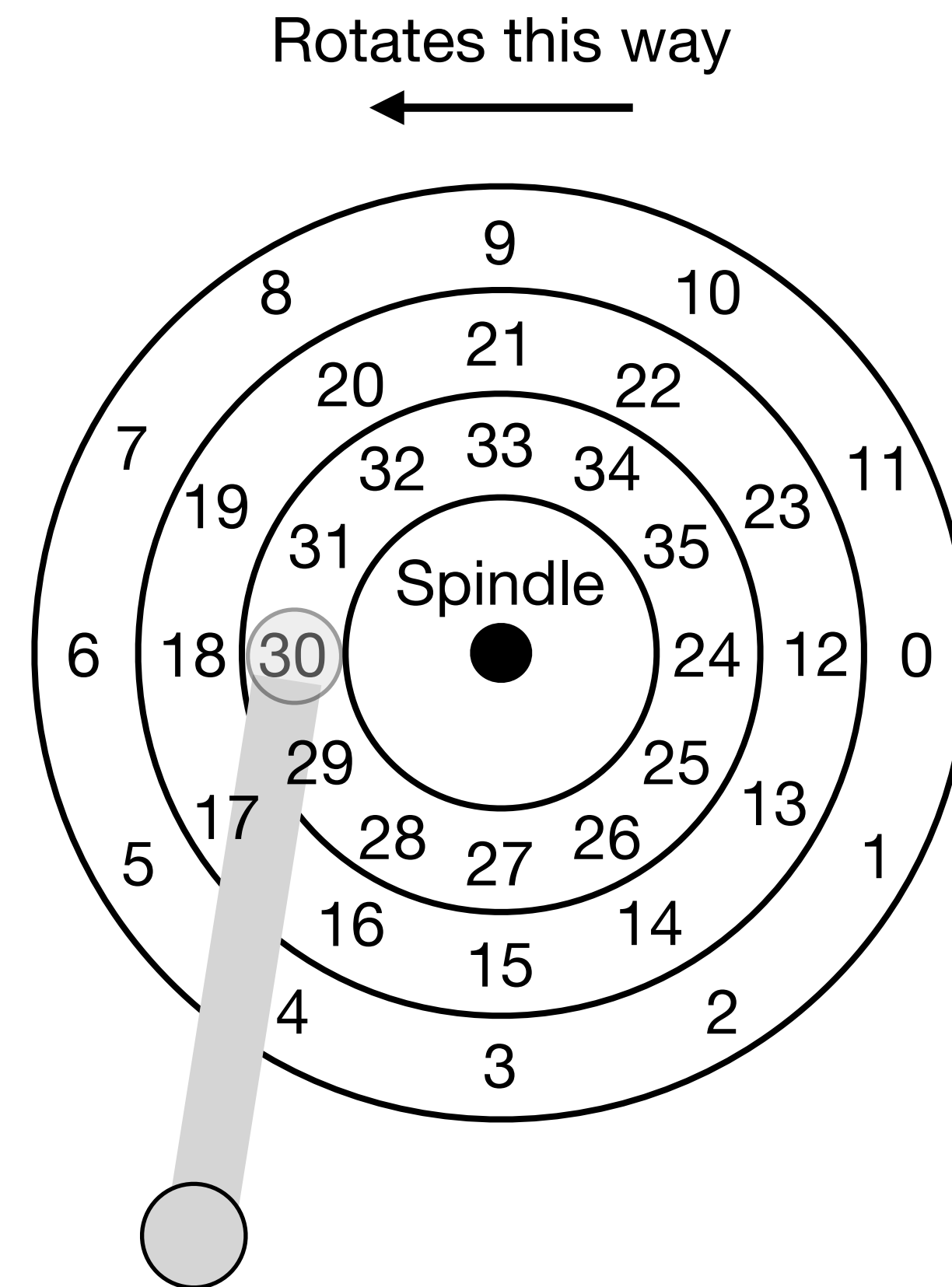
- **Selection criteria:** Serve first the requests for the nearest track (shortest seek time)
- **Example:** Head on sector 30 (inner track), with pending requests for sectors 2 and 21
 - **Decision:** First serve sector 21 (middle track) and then 2 (outer track)
- **Implementation issue:** The drive geometry (disposition of sectors across tracks) is not known by the OS (it just sees an array of sectors)
 - **Solution:** The OS uses a **nearest-block-first (NBF)** policy to schedule requests. Nearest blocks in our sector array will correspond to near tracks at the disk geometry
- **Main Drawback:** SSTF suffers from **starvation**
 - Imagine a steady stream of requests coming to the inner track. Requests from other tracks will never be served!



Scheduling Policies for HDDs

Elevator (aka SCAN or C-SCAN)

- ◎ **SCAN:** Move across the disk from inner to outer tracks (or the other way around) i.e., do a disk **sweep**
 - If a request comes for a sector that has already been passed through on this sweep, queue it for the next sweep (**avoids starvation of far-away requests**)
 - In SCAN, sweeps are performed in both directions (e.g., from inner to outer tracks, then from outer to inner tracks, and then repeat...).
- ◎ **Interesting fact:** SCAN is also known as the **elevator algorithm** due to its sweep strategy. Imagine going from floor 10 to 1, somebody getting in at floor 3 and pressing 4, and the elevator going to the 4th floor because it is closer...
- ◎ **Circular SCAN (C-SCAN):** Sweeps disk tracks in a single direction (e.g., sweep from outer to inner tracks, reset, and start again at the outer tracks)
 - SCAN sweeps tracks back and forth, favoring middle tracks (after serving the outer track, it passes through the middle track twice before returning to the outer one)
 - C-SCAN, a variant of SCAN, is a bit **fairer for inner and outer tracks**



Shortest Positioning Time First (SPTF)

- So far, our algorithms **only consider seek time** (ignoring rotation time)
 - **Example:** Head on sector 30, with requests pending for sectors 16 and 8
 - Which request should be serviced first? It depends...
- If **seek time is high** and **rotation time is small**, then SSTF is a nice approach (i.e., **sector 16** should be served first)
- What if **the rotation time is high and the seek time is small**?
 - Serving **sector 8** first would be faster because the disk arm and head need to move less. Serving sector 16 first would require almost a full disk rotation for the head to be over this sector
- Seek and rotation times are roughly equivalent in modern disks, making sense to consider both to choose the **shortest positioning time**
 - As the OS may not have the necessary knowledge to implement the SPTF algorithm (i.e., the position of the head, track boundaries, ...), this strategy is usually implemented in the disk drive (at the **drive's controller**)

